Green University of Bangladesh
Department of Computer Science and Engineering (CSE)

Faculty of Sciences and Engineering

Semester: (Spring , Year:2026), B.Sc. in CSE (Day)

LAB REPORT NO: 4

Course Title: Artificial intelligence

Course Code: CSE 316 **Section:** 232-D5

+

Lab Experiment Name: Implementation of Minimax and Alpha-Beta Pruning Algorithm

Student Details

	Name	ID
1.	Rukonuzzaman Topu	232002280

Submission Date : **4/12/2026**

Course Teacher's Name : Babe Sultana

[For Teachers use only: Don't Write Anything inside this box]

Lab Report Status

Marks: …… **Signature:.....**

Comments:..... **Date:.....**

1. Introduction:

Title: Implementation of Minimax and Alpha-Beta Pruning Algorithm

Problem Summary: The task focuses on improving decision-making in two-player games using Artificial Intelligence. The Minimax algorithm is commonly used to determine the optimal move by exploring all possible game states in a binary tree structure, where one player (MAX) tries to maximize the score and the other player (MIN) tries to minimize it.

However, a major limitation of Minimax is that it evaluates all nodes in the game tree, which leads to high computational cost, especially for large trees.

To address this issue, Alpha-Beta Pruning is introduced as an optimization technique. It reduces the number of nodes that need to be evaluated by eliminating branches that cannot influence the final decision. This is done using two values:

Alpha (α): the best value that MAX can achieve
Beta (β): the best value that MIN can achieve

The problem requires implementing both Minimax and Alpha-Beta Pruning algorithms on the same binary tree, comparing their performance by counting:

Total nodes evaluated
Nodes pruned (in Alpha-Beta)

Finally, the goal is to demonstrate that Alpha-Beta Pruning improves efficiency while still producing the same optimal result as the Minimax algorithm.

2. Objectives/Aim

- To understand the working principle of the Minimax algorithm in game trees.
- To implement Minimax for optimal decision-making in a two-player environment.
- To apply Alpha-Beta Pruning to reduce unnecessary computations.
- To compare Minimax and Alpha-Beta based on nodes explored.
- To count and analyze the number of nodes visited and pruned.
- To evaluate performance improvement achieved through pruning.
- To verify that both methods produce the same optimal result.

3. Procedure

1. Take input of leaf node values from the user.
2. Check whether the number of leaf nodes is a power of 2.
3. Calculate the depth of the binary tree using $\log_2(n)$.
4. Implement the Minimax algorithm to evaluate the tree recursively.
5. Count the total number of nodes visited during Minimax execution.
6. Implement the Alpha-Beta Pruning algorithm with α and β values.
7. Apply pruning whenever the condition ($\alpha \geq \beta$) is satisfied.
8. Count both visited nodes and pruned nodes in Alpha-Beta.
9. Display the optimal value obtained from both algorithms.
10. Calculate the efficiency improvement percentage.
11. Print all results clearly for comparison.

4.Implementation:

Code :

```
import math


# ---------------- MINIMAX ----------------

def minimax(depth, node_index, is_max, values, max_depth, counter):

    counter["visited"] += 1


    if depth == max_depth:

        return values[node_index]


    if is_max:
```

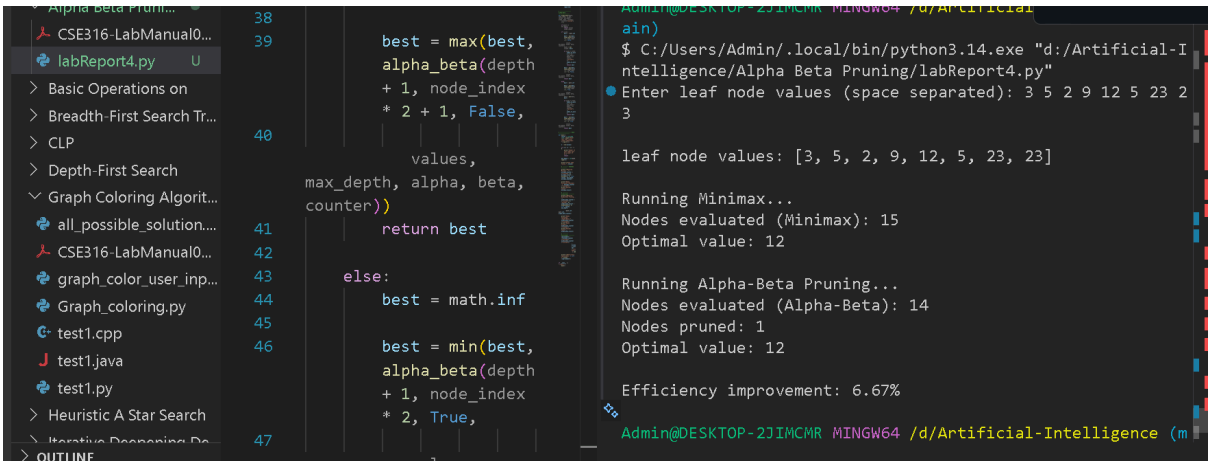
```
        left_val = minimax(depth + 1, node_index * 2, False, values, max_depth, counter)

            right_val = minimax(depth + 1, node_index * 2 + 1, False, values, max_depth, counter)

        return max(left_val, right_val)

    else:

        left_val = minimax(depth + 1, node_index * 2, True, values, max_depth, counter)

            right_val = minimax(depth + 1, node_index * 2 + 1, True, values, max_depth, counter)

        return min(left_val, right_val)




# ---------------- ALPHA-BETA ----------------

def alpha_beta(depth, node_index, is_max, values, max_depth, alpha, beta, counter):

    counter["visited"] += 1


    if depth == max_depth:

        return values[node_index]


    if is_max:

        best = -math.inf


        best = max(best, alpha_beta(depth + 1, node_index * 2, False,

                                    values, max_depth, alpha, beta, counter))

        alpha = max(alpha, best)
```

```
    if alpha >= beta:
        nodes_to_prune = 2 ** (max_depth - depth - 1)
        counter["pruned"] += nodes_to_prune
        return best

    best = max(best, alpha_beta(depth + 1, node_index * 2 + 1, False,
                                values, max_depth, alpha, beta, counter))
    return best

else:
    best = math.inf

    best = min(best, alpha_beta(depth + 1, node_index * 2, True,
                                values, max_depth, alpha, beta, counter))
    beta = min(beta, best)

    if alpha >= beta:
        nodes_to_prune = 2 ** (max_depth - depth - 1)
        counter["pruned"] += nodes_to_prune
        return best

    best = min(best, alpha_beta(depth + 1, node_index * 2 + 1, True,
                                values, max_depth, alpha, beta, counter))
    return best
```

```
# ---------------- MAIN ----------------

def main():

    values = list(map(int, input("Enter leaf node values (space separated): ").split()))


    n = len(values)


    if (n & (n - 1)) != 0:

        print("Number of leaf nodes must be a power of 2!")

        return


    max_depth = int(math.log2(n))


    print("\nleaf node values:", values)


    # -------- MINIMAX --------

    print("\nRunning Minimax...")

    minimax_counter = {"visited": 0}

    minimax_result = minimax(0, 0, True, values, max_depth, minimax_counter)


    print("Nodes evaluated (Minimax):", minimax_counter["visited"])

    print("Optimal value:", minimax_result)
```

```
    # -------- ALPHA-BETA --------
    print("\nRunning Alpha-Beta Pruning...")
    alpha_beta_counter = {"visited": 0, "pruned": 0}
    alpha_beta_result = alpha_beta(0, 0, True, values, max_depth,
                                   -math.inf, math.inf, alpha_beta_counter)

    print("Nodes evaluated (Alpha-Beta):", alpha_beta_counter["visited"])
    print("Nodes pruned:", alpha_beta_counter["pruned"])
    print("Optimal value:", alpha_beta_result)

    # -------- EFFICIENCY --------
    improvement = ((minimax_counter["visited"] - alpha_beta_counter["visited"])
                   / minimax_counter["visited"]) * 100

    print("\nEfficiency improvement: {:.2f}%".format(improvement))


if __name__ == "__main__":
    main()
```

Output:



Analysis and Discussion

The implementation demonstrates a clear difference between the performance of the Minimax algorithm and Alpha-Beta Pruning. In the Minimax approach, all nodes of the binary tree are explored to determine the optimal value, which results in a higher number of node evaluations and increased computational cost.

On the other hand, Alpha-Beta Pruning significantly reduces the number of nodes evaluated by eliminating branches that do not affect the final outcome. This is achieved using the α (alpha) and β (beta) values, which help in identifying and skipping unnecessary computations through pruning.

From the results, it is observed that:

- Both algorithms produce the **same optimal value**, ensuring correctness.
- Alpha-Beta Pruning evaluates **fewer nodes** compared to Minimax.
- A noticeable number of nodes are **pruned**, reducing execution time.
- The **efficiency improvement** confirms that Alpha-Beta is more optimized.

Overall, the experiment proves that Alpha-Beta Pruning enhances the performance of the Minimax algorithm without compromising accuracy, making it suitable for complex decision-making problems in AI.

between uninformed and informed search algorithms, and all objectives were successfully achieved.